SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	y.hema sai lahari1	**Reg. No.:**	192311446
Date:	21-08-2026	**Duration:**	60 minutes

Code of Conduct

I y.hema sai lahari (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:y.hema sai lahari

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. **Repository Organization**
    - Evaluate the repository structure, folder organization, and file naming conventions.
    - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
    - Review the source code for readability, modularity, coding standards, and documentation.
    - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
    - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
    - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
    - Demonstrate that the application functions correctly through appropriate testing.
    - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
    - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history

- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

	management practices.			
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentatio n.
Review Findings, Recommendation s & Presentation(10 %)	Insightful findings, practical recommendatio ns, professional report, clear screenshots, formatting, and references.	Good recommendatio ns and presentation.	Basic recommendatio ns with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Project Title

Temple Donation and Event Management System

---

1. Problem Overview

1.1 Problem Statement

The **Temple Donation and Event Management System** is a web-based application developed to digitally manage temple donations, donor information, temple events, and donation records. Traditional temple management systems often depend on manual registers and paper-based receipts, making it difficult to maintain accurate records and generate reports.

The proposed system provides a centralized platform for administrators and donors. It allows temple administrators to maintain donor details, record donations, manage temple events, and view donation reports.

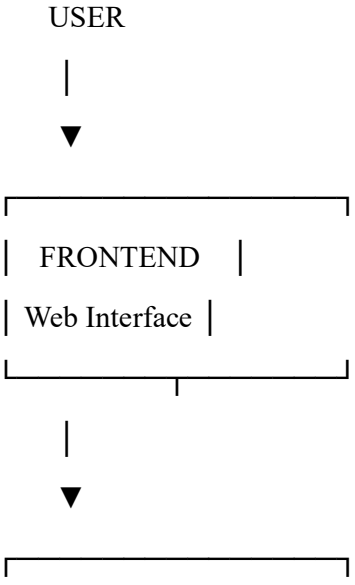
The system improves the efficiency, accuracy, transparency, and maintainability of temple management activities.

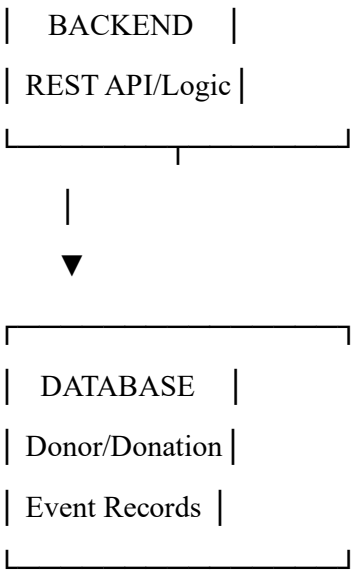
1.2 Implemented Solution

The system is implemented as a modular web application consisting of frontend, backend, and database components.

The frontend provides the user interface, while the backend handles business logic and API requests. The database stores donor, donation, event, and user information.

Basic Architecture





1.3 Project Objectives

The main objectives are:

1. To digitize temple donation management.
2. To maintain centralized donor records.
3. To manage temple donations efficiently.
4. To organize temple events and festivals.
5. To provide donation history and reports.
6. To reduce paperwork and manual errors.
7. To improve data accessibility.
8. To provide a maintainable and scalable software solution.

1.4 Key Functionalities

Donor Management

- Register new donors.
- Update donor information.
- View donor details.
- Search donor records.

Donation Management

- Record donations.
- Store donation amount and date.

- Associate donations with donors.
- View donation history.
- Generate donation reports.

Event Management

- Create temple events.
- Update event information.
- Delete events.
- View upcoming events.
- Maintain event schedules.

Admin Management

- Admin authentication.
- Manage donors.
- Manage donations.
- Manage events.
- View reports.

Reporting

- Daily donation reports.
- Monthly donation reports.
- Donor-wise reports.
- Event-wise donation information.

---

2. Repository Organization

2.1 Repository Structure

The project repository follows a modular structure:

Temple-Donation-Management/

│

├── frontend/

│ ├── public/

│ ├── src/

```
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── assets/
│   │   └── App.js
│   ├── package.json
│   └── Dockerfile
│
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middleware/
│   ├── config/
│   ├── server.js
│   ├── package.json
│   └── Dockerfile
│
├── database/
│   └── init/
│
├── docs/
│   ├── architecture/
│   └── screenshots/
│
├── .gitignore
├── docker-compose.yml
└── README.md
```

2.2 Folder Evaluation

Frontend

The frontend folder contains all user interface-related code. Separating frontend code makes it easier to modify the user interface without affecting backend logic.

Backend

The backend folder contains server-side application logic, APIs, authentication, controllers, and database operations.

Controllers

Controllers handle application operations such as creating donations, retrieving donor records, and managing events.

Models

Models define the structure of database records such as:

- Donor
- Donation
- Event
- User

Routes

Routes define API endpoints for communication between the frontend and backend.

Middleware

Middleware handles common processing such as authentication and request validation.

Database

The database folder contains database-related scripts and initialization files.

Docs

The docs folder stores architecture diagrams, screenshots, and project documentation.

2.3 File Naming Conventions

The project follows descriptive naming conventions.

Examples:

donorController.js

donationController.js

eventController.js

donorModel.js

donationModel.js

eventModel.js

donationRoutes.js

eventRoutes.js

Descriptive names make files easier to locate and understand.

2.4 Maintainability and Collaboration

The repository structure supports maintainability because related files are grouped together. Developers can independently work on frontend, backend, database, or documentation components.

The modular structure also reduces the possibility of unnecessary changes to unrelated modules.

---

3. Code Quality Review

3.1 Readability

The source code should use meaningful variable, function, and file names.

Example:

```
const donationAmount = 5000;

const donorName = "Ravi Kumar";


function createDonation(donationData) {

   // Process donation

}
```

Names such as donationAmount, donorName, and createDonation() clearly describe their purpose.

3.2 Modularity

The application is divided into independent modules such as:

```
Authentication
    │
    ├── Donor Management
    │
```

├── Donation Management

│

├── Event Management

│

└── Reporting

Modular design makes the system easier to test, modify, and extend.

3.3 Coding Standards

The following coding practices are recommended:

- Use meaningful variable names.
- Use consistent indentation.
- Avoid unnecessary duplicate code.
- Keep functions small and focused.
- Use comments where complex logic requires explanation.
- Follow consistent naming conventions.
- Validate user input.
- Handle errors properly.

3.4 Documentation in Code

Important functions should contain comments or documentation.

Example:

```
// Creates a new donation record
function createDonation(donationData) {
   // Donation processing logic
}
```

Documentation helps another developer understand the purpose of the function.

3.5 Strengths

The major strengths identified during code review are:

1. Modular project structure.
2. Separation of frontend and backend.
3. Meaningful file organization.

4. Reusable components.
5. Centralized database access.
6. Git-based version control.
7. Docker-based deployment support.
8. Clear separation of application responsibilities.

3.6 Areas for Improvement

The following improvements can be made:

1. Increase automated test coverage.
2. Add stronger input validation.
3. Improve error-handling mechanisms.
4. Use consistent coding standards throughout the project.
5. Add API documentation.
6. Reduce duplicate code.
7. Improve security of authentication.
8. Add logging and monitoring.
9. Add automated code quality checks.

---

4. Version Control Evaluation

4.1 Git Repository Analysis

Git is used to maintain the source code and track changes throughout the development process.

The repository contains multiple commits representing different development activities.

Example:

Initial project setup

↓

Create frontend

↓

Create backend API

↓

Add donor management

↓

Add donation management

↓

Add event management

↓

Add reporting

↓

Add Docker configuration

↓

Update documentation

4.2 Commit History

The following command can be used to inspect the commit history:

git log --oneline --graph --all

Example output:

a82f31c Add Docker configuration

b73e20a Add event management module

c61d45b Add donation management module

d52c19e Add donor management module

e41a28f Create backend API

f30b17a Initial project setup

4.3 Commit Message Evaluation

The commit messages are descriptive and indicate the purpose of each change.

Good examples:

Add donor management module

Add donation validation

Implement event management

Fix donation API error

Update project README

Add Docker Compose configuration

Poor commit messages such as:

changes

update

final

new

test

should be avoided because they do not clearly explain what was changed.

4.4 Branching Strategy

The project can use:

main

  │

development

  │

  ├── feature/donor-management

  ├── feature/donation-management

  ├── feature/event-management

  └── feature/reporting

The main branch contains stable code, while development contains integrated development work. Feature branches allow developers to work independently.

4.5 Version Control Benefits

Git supports collaborative software development by providing:

- Change tracking.
- Branch management.
- Code backup.
- Collaboration.
- Version recovery.
- Merge support.
- Conflict detection.

- Project history.

---

5. Code Testing and Validation

5.1 Testing Objective

Testing is performed to verify that the Temple Donation and Event Management System functions according to its requirements.

5.2 Test Cases

Test ID	Test Case	Expected Result
TC01	Admin login with valid credentials	Admin dashboard opens
TC02	Admin login with invalid credentials	Error message displayed
TC03	Add new donor	Donor record created
TC04	Update donor	Donor information updated
TC05	Delete donor	Donor removed
TC06	Add donation	Donation record stored
TC07	View donation history	Correct records displayed
TC08	Create temple event	Event created successfully
TC09	Update event	Event information updated
TC10	Delete event	Event removed
TC11	Generate donation report	Correct report displayed
TC12	Database connection test	Application connects successfully

5.3 Sample Testing Procedure

Test Case: Add Donation

Input:

Donor Name: Ravi Kumar

Donation Amount: ₹5000

Donation Date: 21-08-2026

Purpose: Temple Development

Expected Result:

The donation should be successfully stored and displayed in the donation history.

Actual Result:

Donation record is successfully added and displayed.

Status:

PASS

Test Case: Create Temple Event

Input:

Event Name: Annual Temple Festival

Date: 25-08-2026

Location: Main Temple

Expected Result:

The event should be created successfully.

Actual Result:

Event is displayed in the event management section.

Status:

PASS

5.4 Testing Evidence

The following screenshots should be included:

- Admin login screen.
- Admin dashboard.
- Add donor screen.
- Donation entry screen.
- Donation history screen.
- Event creation screen.
- Event list.
- Donation report.
- Successful application execution.
- Database/application output.

---

6. Repository Documentation

6.1 README Evaluation

The project should contain a README.md file that provides essential information about the system.

A good README should include:

1. Project Title

2. Problem Statement

3. Project Description

4. Features

5. Technologies Used

6. Project Structure

7. Installation Instructions

8. Configuration

9. How to Run

10. Testing

11. Docker Instructions

12. Git Workflow

13. Screenshots

14. Future Enhancements

6.2 Installation Instructions

The README should explain how to install the project.

Example:

git clone <repository-url>

cd Temple-Donation-Management

cd backend

npm install

cd ../frontend

npm install

6.3 Running the Application

The application can be started using:

npm start

or through Docker:

docker compose up --build

6.4 Dependencies

The README should list the major dependencies used by the application.

Example:

Frontend:

- React
- Axios
- React Router

Backend:

- Node.js
- Express.js
- Database Driver

Development:

- Git
- Docker
- Docker Compose

6.5 Documentation Improvements

The documentation can be improved by:

1. Adding screenshots.
2. Providing complete installation instructions.
3. Adding API documentation.

4. Providing sample credentials for testing where appropriate.
5. Adding troubleshooting instructions.
6. Documenting database configuration.
7. Adding Docker instructions.
8. Maintaining a changelog.

---

7. Code Review Findings and Recommendations

7.1 Overall Findings

The code review shows that the Temple Donation and Event Management System follows a modular architecture and separates major application responsibilities.

The repository structure is organized into frontend, backend, database, and documentation components. Git provides effective version control and enables feature-based development.

The major functional modules include donor management, donation management, event management, authentication, and reporting.

7.2 Code Quality Findings

Area	Finding	Recommendation
Readability	Mostly clear naming	Maintain naming standards
Modularity	Components are separated	Further improve reusable modules
Documentation	Basic documentation available	Add detailed API documentation
Error Handling	Basic handling	Add centralized error handling
Validation	Input validation required	Add server-side validation
Testing	Functional testing available	Add automated tests
Security	Authentication required	Improve authorization and security
Maintainability	Modular structure	Continue modular architecture

7.3 Repository Recommendations

The following repository improvements are recommended:

1. Maintain a clean folder structure.
2. Use meaningful commit messages.
3. Use feature branches for development.

4. Protect the main branch.
5. Review code before merging.
6. Keep sensitive files outside Git.
7. Maintain an updated README.
8. Add automated CI/CD.
9. Use issue tracking for bugs and enhancements.
10. Tag stable releases.

7.4 Future Enhancements

The system can be enhanced with:

Online Payment Integration

Allow devotees to make donations through secure online payment gateways.

QR Code Receipts

Generate QR-code-based digital donation receipts.

SMS and Email Notifications

Automatically notify donors after successful donations.

Advanced Analytics

Display donation trends using charts and dashboards.

Mobile Application

Develop an Android/iOS application for devotees and temple administrators.

Cloud Deployment

Deploy the application using cloud infrastructure.

Automated CI/CD

Automatically build, test, and deploy the application whenever approved changes are pushed to the repository.

Backup and Recovery

Implement automatic database backup and recovery mechanisms.

Conclusion

The **Temple Donation and Event Management System** provides a structured software solution for managing temple donors, donations, events, and reports. The code review

indicates that a modular repository structure, meaningful naming conventions, Git-based version control, and proper documentation contribute significantly to project maintainability.

Testing validates the major functions of the application, while Git supports collaborative development through commits and branches. Further improvements in automated testing, security, documentation, CI/CD, cloud deployment, and analytics can make the system more reliable and scalable.

Overall, the project demonstrates effective application of software engineering practices and provides a strong foundation for future development.